

Analizador Léxico para a Linguagem C-: Projeto de Implementação utilizando Máquina de Moore

Gabriela Smith Ferreira - a2482258

¹Departamento Acadêmico de Computação (DACOM)
Universidade Tecnológica Federal do Paraná (UTFPR)

Resumo

Este trabalho tem como objetivo o desenvolvimento de um analisador léxico para a linguagem C-, utilizando autômatos finitos, mais especificamente uma Máquina de Moore. A ferramenta gerará uma lista de tokens identificados a partir do código-fonte da linguagem, realizando a conversão de caracteres em tokens. A implementação se baseia em expressões regulares e autômatos finitos determinísticos para realizar a análise léxica.

1 Introdução

A análise léxica é a fase inicial de um compilador e consiste no processo de ler o código-fonte e dividir o texto em unidades semânticas chamadas de tokens. Neste trabalho, estamos desenvolvendo um analisador léxico para a linguagem C-, uma versão simplificada da linguagem C, utilizando autômatos finitos. O analisador será responsável por identificar os tokens presentes no código-fonte, como palavras-chave, operadores lógicos, aritméticos e relacionais, nomes de variáveis e funções, e números, para então gerar uma lista de tokens.

A implementação do analisador léxico será baseada em uma Máquina de Moore, um tipo de autômato finito que gera uma saída com base nos estados e nas transições. Utilizaremos expressões regulares para especificar os padrões de tokens da linguagem C-, como palavras-chave (exemplo: 'if', 'int', 'return'), operadores (como '+', '-', '*', '/', '&&'), delimitadores (como ';', '(', ')'), além de identificadores e números. Essas expressões regulares serão convertidas em um autômato finito determinístico, que será a base para o nosso analisador léxico.

O principal objetivo deste trabalho é construir a primeira fase de um compilador para a linguagem C-.

2 Objetivo

O objetivo deste trabalho é projetar e implementar um analisador léxico para a linguagem C-, utilizando autômatos finitos e máquinas de Moore. A ferramenta será capaz de identificar tokens do código-fonte, gerando uma lista de tokens que servirá de entrada para as fases posteriores do compilador, como a análise sintática e semântica.

3 Metodologia

A abordagem adotada para a implementação do analisador léxico é a utilização de autômatos finitos determinísticos (DFA). A partir de expressões regulares que definem os tokens da linguagem

C-, o DFA será construído e implementado. O código-fonte será lido caractere por caractere, e as transições de estado do autômato serão simuladas, identificando os tokens correspondentes.

Após a construção do DFA, implementaremos a função de análise léxica que será responsável por realizar a varredura do código-fonte, identificando os tokens e gerando uma lista com as correspondências encontradas.

4 Resultados Esperados

Espera-se que o analisador léxico seja capaz de processar um código-fonte da linguagem C- e gerar uma lista precisa de tokens. A ferramenta deverá ser eficiente e capaz de lidar com diferentes combinações de tokens no código, identificando corretamente as palavras-chave, operadores, identificadores e números.

5 O Analisador Léxico

5.1 Introdução à Análise Léxica

A análise léxica, também conhecida como varredura, é a primeira fase do processo de compilação de um programa. Ela tem como principal objetivo transformar o código-fonte, composto por uma sequência de caracteres, em uma sequência de tokens, que são unidades léxicas significativas para o compilador. Esses tokens representam as construções básicas da linguagem de programação, como palavras-chave, operadores, identificadores, números e outros símbolos.

Durante a análise léxica, o código-fonte é processado para identificar essas unidades, com base em padrões predefinidos que são representados por expressões regulares. Para cada padrão reconhecido, o analisador léxico gera um token correspondente, que é passado para a próxima fase do compilador, a análise sintática.

A fase de análise léxica é composta por 2 fases principais:

- **Leitura do código-fonte:** O primeiro passo envolve a leitura do código-fonte do programa, onde o texto é analisado caractere por caractere, em busca de padrões que correspondam aos tokens.
- **Reconhecimento de tokens:** O analisador léxico utiliza expressões regulares para identificar tokens específicos que incluem, por exemplo, palavras-chave como `if`, `while`, `int`, e operadores aritméticos como `+` e `-`.

5.2 Implementação do Analisador Léxico

Nesta seção, apresentamos a implementação do analisador léxico, responsável por transformar o código-fonte de entrada em uma sequência de tokens. O analisador é baseado em uma máquina de Moore e inclui um sistema de tratamento de erros.

Quanto a estrutura do analisador léxico, ele é implementado no arquivo `analex.py`. Ele utiliza a classe `Moore` para representar a máquina de estados finitos que reconhece tokens. O processamento dos tokens ocorre através da função `process_tokens`, que recebe o código-fonte como entrada e retorna a sequência de tokens identificados.

Abaixo está a definição dos estados e símbolos utilizados na máquina de Moore:

- **Estados:** A máquina de Moore possui múltiplos estados (`q0`, `q1`, `q2`, ..., `q64`) responsáveis por processar diferentes partes do código-fonte.

- **Símbolos:** Os caracteres reconhecidos incluem letras, números, operadores (+, -, *, /, <, >) e símbolos especiais ((), {}, ;).
- **Tokens:** Após o reconhecimento, os tokens são classificados em categorias como palavras-chave (INT, IF, ELSE), identificadores (ID), e números (NUMBER).

O analisador léxico também verifica a existência do arquivo de entrada e trata erros de uso, garantindo que apenas arquivos no formato correto sejam processados.

5.2.1 Testes do Analisador Léxico

O arquivo `analex_test.py` contém testes automatizados para validar a execução do analisador léxico. Ele utiliza a biblioteca `pytest` e executa o analisador em diversos arquivos de teste. O teste verifica se a saída gerada corresponde à saída esperada, armazenada nos arquivos `.lex.out`.

A execução dos testes ocorre da seguinte forma:

1. O script busca arquivos de teste no diretório `tests/`.
2. Para cada arquivo de entrada, o analisador léxico é executado.
3. A saída gerada é comparada com a saída esperada.
4. Se houver discrepâncias, o teste falha, indicando um erro na análise léxica.

Os testes podem ser executados de 3 maneiras:

1. **Arquivo por arquivo detalhadamente:** Ao executar o comando `py analex.py ./tests/nome_do_arquivo` no terminal serão exibidos todos os estados, outputs, transições e alfabeto do autômato, além, claro, da saída de tokens esperada.
2. **Arquivo por arquivo sem detalhes:** Ao executar o comando `py analex.py -k ./tests/nome_do_arquivo` no terminal será exibida apenas a lista de tokens.
3. **Todos os arquivos sem detalhes:** Ao executar o comando `pytest -v` no terminal serão exibidos todos os testes e se eles passaram ou não.

OBSERVAÇÃO: Todos os testes foram executados apenas no Windows, para Linux deve-se substituir "py" por "python3" na execução dos comandos.

Segue abaixo o código que executa os testes:

```

1 test_cases = [("", "-k"), ("teste.c", "-k"), ("notexists.cm", "-k")]
2 for file in fnmatch.filter(os.listdir('tests'), '*.cm'):
3     test_cases.append((file, "-k"))
4
5 @pytest.mark.parametrize("input_file, args", test_cases)
6 def test_execute(input_file, args):
7     if input_file != '':
8         path_file = 'tests/' + input_file
9     else:
10        path_file = ""
11
12    cmd = "python analex.py {0} {1}".format(args, path_file)
13    process = subprocess.Popen(shlex.split(cmd), stdout=subprocess.PIPE, stderr=
14        subprocess.PIPE)
15
16    stdout, stderr = process.communicate()
17    output_lines = stdout.decode("utf-8").strip().splitlines()
18    generated_output = "\n".join(output_lines)

```

```

19 path_file = 'tests/' + input_file
20 expected_output_file = path_file + ".lex.out"
21
22 if os.path.exists(expected_output_file):
23     with open(expected_output_file, "r") as output_file:
24         expected_output = output_file.read().strip()
25 else:
26     expected_output = stderr.decode("utf-8").strip()
27
28 assert generated_output.strip() == expected_output.strip()

```

5.2.2 Tratamento de Erros

O tratamento de erros no analisador léxico é realizado pelo módulo `myerror.py`. Ele utiliza a biblioteca `configparser` para carregar mensagens de erro a partir de um arquivo externo (`ErrorMessages.properties`). A classe `MyError` permite a geração de mensagens de erro detalhadas, incluindo informações como linha e coluna do erro.

O método `newError` constrói a mensagem no seguinte formato: `[key]=[Value]`

Existem quatro possibilidades de erros:

```

ERR-LEX-USE=Uso: python analex.py file.cm
ERR-LEX-NOT-CM=Não é um arquivo .cm.
ERR-LEX-FILE-NOT-EXISTS=Arquivo .cm não existe.
ERR-LEX-INV-CHAR=Caracter inválido.

```

Gostaria de registrar que, até o momento, o analisador léxico não possui tratamento completo para todos os possíveis estados e caracteres. Alguns estados e caracteres podem não ser reconhecidos corretamente devido à falta de implementações e transições no autômato. Isso pode resultar em erros durante a análise de códigos-fonte que contêm casos de transições não tratadas.

É importante destacar que esses erros são conhecidos, pois não fui capaz de pensar na lógica necessária para tratar cenários específicos e cobrir todos todas as possibilidades de entrada.

Os erros conhecidos são:

1. O autômato não é capaz de reconhecer qualquer ID que inicia com a mesma letra de uma palavra-chave. Por exemplo: `int v` imprimirá `INT`.
2. Na mesma lógica do primeiro erro, não é capaz de imprimir o caracter após o ID que inicia com a mesma letra de uma palavra-chave. Por exemplo: `(u+v)`; imprimirá `LPAREN`, `ID`, `PLUS`, `SEMICOLON`.
3. O autômato não é capaz de ignorar caracteres de comentários, ou seja, testes que possuem `/* COMENTÁRIO */` continuará imprimindo todos os caracteres.

5.3 Diagrama do Autômato

O analisador léxico desenvolvido é baseado em um Autômato Finito Determinístico (AFD), que processa a entrada caractere por caractere e identifica os tokens correspondentes. Esse autômato é representado por um conjunto de estados e transições, cada um associado a uma categoria léxica específica, como palavras-chave, operadores e identificadores.

O diagrama abaixo ilustra a estrutura do autômato utilizado no analisador léxico, destacando os estados e as transições entre eles conforme os caracteres são lidos. Esse modelo permite uma

análise eficiente do código-fonte, garantindo que cada lexema seja corretamente reconhecido e classificado.

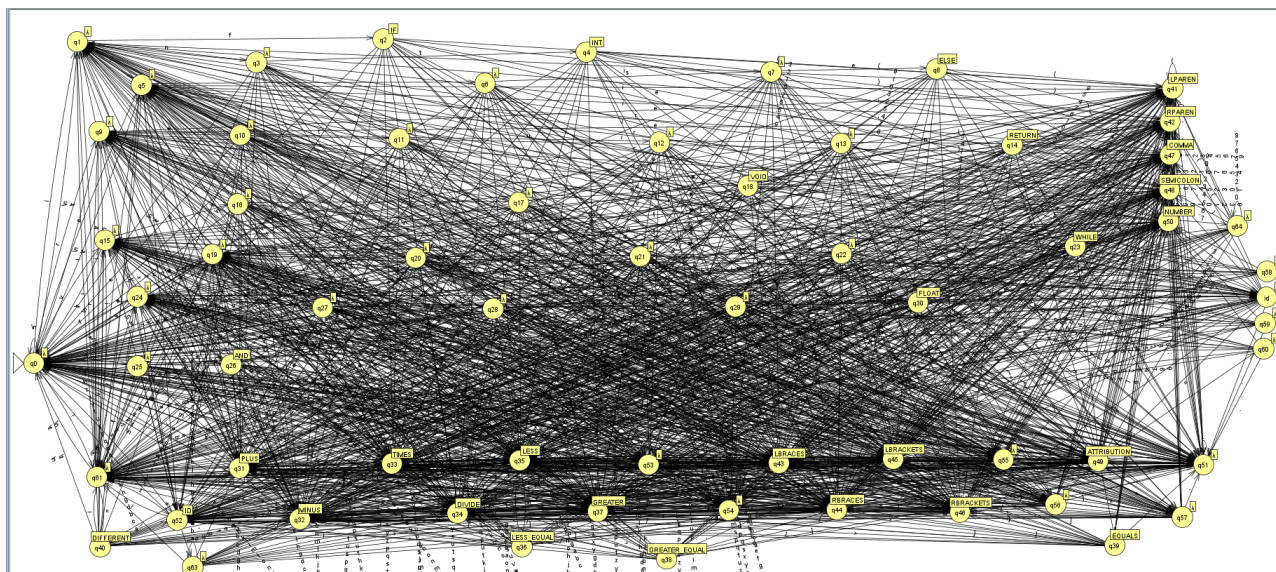


Figura 1: Máquina de Moore

Para ilustrar o funcionamento do analisador léxico, apresento exemplos de entrada e saída do sistema. A entrada consiste em uma lista de caracteres que representam um código-fonte simples, enquanto a saída exibe a lista de tokens reconhecidos pelo analisador.

Na entrada, cada palavra-chave, identificador, número e operador será processado pelo autômato e classificado de acordo com sua categoria léxica. A saída gerada apresenta esses tokens juntamente com suas respectivas classificações, demonstrando como o analisador léxico interpreta o código-fonte.

A seguir, apresento imagens exemplificando esse processo. As imagens exibem a entrada e saída respectiva dos testes `prog-000.cm` ao `prog-007.cm` em ordem crescente. A impressão em lista ocorre apenas via terminal.

Input	Result
int	INT

Figura 2: Teste prog-000.cm

input	result
int	INT
main(void){	IDL PARENVOIDR PAREN LBRACES
}	RBRACES

Figura 3: Teste prog-001.cm

Input	Result
int	INT
main(void){	IDLPARENVOIDRPARENLBRACES
return(0);	RETURNLPARENNUMBERRPARENSEMICOLON
}	RBRACES

Figura 4: Teste prog-002.cm

Input	Result
int	INT
a;	IDSEMICOLON
float	FLOAT
b;	IDSEMICOLON
int	INT
main(void){	IDL PAREN VOID R PAREN L BRACES
int	INT
x;	IDSEMICOLON
float	FLOAT
y;	IDSEMICOLON
return(0);	RETURN L PAREN NUMBER R PAREN SEMICOLON
}	R BRACES

Figura 5: Teste prog-003.cm

Input	Result
int	INT
soma	ID
(int	L PAREN INT
u,	ID COMMA
int	INT
v){	L BRACES
return	RETURN
u	ID
+	PLUS
v;	
}	R BRACES
int	INT
main(void){	IDL PAREN VOID R PAREN L BRACES
int	INT
x;	IDSEMICOLON
int	INT
y;	IDSEMICOLON
int	INT
z;	IDSEMICOLON
x	ID
=	ATTRIBUTION
input();	IDL PAREN R PAREN SEMICOLON
y	ID
=	ATTRIBUTION
input();	IDL PAREN R PAREN SEMICOLON
z	ID
=	ATTRIBUTION
soma(x,y);	IDL PAREN ID COMMA ID R PAREN SEMICOLON
output(z);	IDL PAREN ID R PAREN SEMICOLON
return	RETURN
0;	NUMBERSEMICOLON
}	R BRACES

Figura 6: Teste prog-004.cm

Input	Result
int	INT
v){	LBRACES
if	IF
(v	LPAREN
==	EQUALS
0)	NUMBER RPAREN
return	RETURN
u;	ID SEMICOLON
else	ELSE
return	RETURN
gcd(v,u-u/v*v);	ID LPAREN ID MINUS ID DIVIDE TIMES SEMICOLON
/*	DIVIDE TIMES
u-u/v*v	ID MINUS ID DIVIDE TIMES
==	EQUALS
u	ID
mod	ID
v	
*/	TIMES DIVIDE
}	RBRACES
void	VOID
main(void){	ID LPAREN VOID RPAREN LBRACES
int	INT
x;	ID SEMICOLON
int	INT
y;	ID SEMICOLON
x	ID
=	
input();	ID LPAREN RPAREN SEMICOLON
y	ID
=	
input();	ID LPAREN RPAREN SEMICOLON
output(gcd(x,y));	ID LPAREN ID LPAREN ID COMMA ID RPAREN RPAREN S...
}	RBRACES

Figura 7: Teste prog-005.cm

Input	Result
int	INT
a;	ID SEMICOLON
int	INT
b[10];	ID LBRACKETS NUMBER RBRACKETS SEMICOLON
int	INT
input(void){	ID LPAREN VOID RPAREN LBRACES
return;	RETURN SEMICOLON
}	RBRACES
void	VOID
output(int	ID LPAREN INT
x){	ID RPAREN LBRACES
}	RBRACES
int	INT
func(int	ID LPAREN INT
x,	ID COMMA
int	INT
y){	ID RPAREN LBRACES
int	INT
res;	ID SEMICOLON
res	ID
=	
x	ID
+	PLUS
y	ID
-	MINUS
2;	NUMBER SEMICOLON
if(res	IF LPAREN ID
!=	DIFFERENT

Figura 8: Teste prog-006.cm

!=	DIFFERENT
0){	NUMBER RPAREN LBRACES
res	ID
=	
-1;	MINUS NUMBER SEMICOLON
}	RBRACES
return(res);	RETURN LPAREN ID RPAREN SEMICOLON
}	RBRACES
int	INT
main(void){	ID LPAREN VOID RPAREN LBRACES
a	ID
=	
input();	ID LPAREN RPAREN SEMICOLON
b	ID
=	
input();	ID LPAREN RPAREN SEMICOLON
b[0]	ID LBRACKETS NUMBER RBRACKETS
=	
a;	ID SEMICOLON
b[1]	ID LBRACKETS NUMBER RBRACKETS
=	
func(a,b);	ID LPAREN ID COMMA ID RPAREN SEMICOLON
output(a);	ID LPAREN ID RPAREN SEMICOLON
return(0);	RETURN LPAREN NUMBER RPAREN SEMICOLON
}	RBRACES

Figura 9: Teste prog-006.cm

Input	Result
int	INT
a,	IDSEMICOLON
int	INT
b[10],	IDLBACKETSNUMBERRBRACKETSSSEMICOLON
int	INT
c[3][5],	IDLBACKETSNUMBERRBRACKETSLBRACKETSNUMBERRBR...
b	ID
=	ATTRIBUTION
10	NUMBER
int	INT
func(int	IDLPARENINT
x,	IDCOMMA
int	INT
y){	IDRPARENLBRACES
int	INT
res;	IDSEMICOLON
res	ID
=	ATTRIBUTION
x	ID
+	PLUS
y,	IDSEMICOLON
return(res);	RETURNLPARENIDRPARENSEMICOLON
}	RBRACES
int	INT
main(){	IDLPARENRPARENLBRACES
a	ID
=	ATTRIBUTION
input(),	IDLPARENRPARENSEMICOLON
b	ID
=	ATTRIBUTION
input(),	IDLPARENRPARENSEMICOLON

Figura 10: Teste prog-007.cm

b	ID
=	ATTRIBUTION
input(),	IDLPARENRPARENSEMICOLON
b[0]	IDLBACKETSNUMBERRBRACKETS
=	ATTRIBUTION
a,	IDSEMICOLON
b[1]	IDLBACKETSNUMBERRBRACKETS
=	ATTRIBUTION
b,	IDSEMICOLON
c[0][1]	IDLBACKETSNUMBERRBRACKETSLBRACKETSNUMBERRBR...
=	ATTRIBUTION
func(a,b);	IDLPARENDCOMMAIDRPARENSEMICOLON
output(c[3]);	IDLPARENDLBACKETSNUMBERRBRACKETS
return(0);	RETURNLPARENNUMBERRPARENSEMICOLON
}	RBRACES

Figura 11: Teste prog-007.cm

6 Conclusão

Este trabalho visa contribuir para o desenvolvimento de compiladores e ferramentas de compilação ao implementar uma fase inicial de análise léxica para a linguagem C-. A utilização de autômatos finitos para a construção do analisador léxico é uma abordagem eficiente e bem estabelecida na teoria de compiladores. Espera-se que a implementação contribua para uma melhor compreensão do processo de análise léxica e sua aplicação na construção de compiladores.